



**NUST COLLEGE OF
ELECTRICAL AND MECHANICAL ENGINEERING**



MEDIBUDDY
A PROJECT REPORT
DE-43 (DC & SE)
Submitted by
NS ABDULLAH UPPAL
NS UZAIR WAHEED
NS MEER MUHAMMAD
NS SEERAT NOOR
BACHELORS
IN
COMPUTER ENGINEERING
YEAR
2025

PROJECT SUPERVISOR
DR. USMAN AKRAM
DR. ASAD MANSOOR KHAN
DR. AYESHA HAUQE

DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING
COLLEGE OF ELECTRICAL & MECHANICAL ENGINEERING
NATIONAL UNIVERSITY OF SCIENCES AND TECHNOLOGY,
ISLAMABAD, PAKISTAN

Certification

This is to certify that NS Abdullah Uppal 381058, NS Uzair Waheed 371828, NS Seerat Noor 374583, and NS Meer Muhammad 393365 have successfully completed the final project "MediBuddy – AI-Powered Diagnostic and Educational Platform" , at the National University of Sciences and Technology (NUST), College of Electrical and Mechanical Engineering , to fulfill the partial requirement of the degree of Computer Engineering.

Signature of Project Supervisor
Dr. Usman Akram
Professor

Signature of Project Supervisor
Dr. Asad Mansoor Khan
Associate Professor

Sustainable Development Goals (SDGs)

SDG No	Description of SDG	SDG No	Description of SDG
SDG 1	No Poverty	SDG 9	Industry, Innovation, and Infrastructure
SDG 2	Zero Hunger	SDG 10	Reduced Inequalities
SDG 3	Good Health and Well Being	SDG 11	Sustainable Cities and Communities
SDG 4	Quality Education	SDG 12	Responsible Consumption and Production
SDG 5	Gender Equality	SDG 13	Climate Change
SDG 6	Clean Water and Sanitation	SDG 14	Life Below Water
SDG 7	Affordable and Clean Energy	SDG 15	Life on Land
SDG 8	Decent Work and Economic Growth	SDG 16	Peace, Justice and Strong Institutions
		SDG 17	Partnerships for the Goals



Sustainable Development Goals

Complex Engineering Problem

Table 1: Range of Complex Problem Solving and Activities

Range of Complex Problem Solving

	Attribute	Complex Problem	
1	Range of conflicting requirements	Involve wide-ranging or conflicting technical, engineering and other issues.	
2	Depth of analysis required	Have no obvious solution and require abstract thinking, originality in analysis to formulate suitable models.	
3	Depth of knowledge required	Requires research-based knowledge much of which is at, or informed by, the forefront of the professional discipline and which allows a fundamentals-based, first principles analytical approach.	
4	Familiarity of issues	Involve infrequently encountered issues	
5	Extent of applicable codes	Are outside problems encompassed by standards and codes of practice for professional engineering.	
6	Extent of stakeholder involvement and level of conflicting requirements	Involve diverse groups of stakeholders with widely varying needs.	
7	Consequences	Have significant consequences in a range of contexts.	
8	Interdependence	Are high level problems including many component parts or sub-problems	

Range of Complex Problem Activities

	Attribute	Complex Activities	
1	Range of resources	Involve the use of diverse resources (and for this purpose, resources include people, money, equipment, materials, information and technologies).	
2	Level of interaction	Require resolution of significant problems arising from interactions between wide ranging and conflicting technical, engineering or other issues.	
3	Innovation	Involve creative use of engineering principles and research-based knowledge in novel ways.	
4	Consequences to society and the environment	Have significant consequences in a range of contexts, characterized by difficulty of prediction and mitigation.	
5	Familiarity	Can extend beyond previous experiences by applying principles-based approaches.	

*Dedicated to our Parents, Teacher and
mentors*

Acknowledgment

First and foremost, we express our deepest gratitude to Allah Almighty, whose guidance and blessings gave us the strength and perseverance to complete this project. We extend our sincere thanks to our supervisor, Dr. Usman Akram, for his invaluable mentorship, consistent encouragement, and insightful feedback throughout this journey. His guidance shaped our direction and kept us focused. We are equally thankful to our co-supervisor, Dr. Asad Khan, whose expert advice and unwavering support played a vital role in refining our approach and overcoming challenges. We also acknowledge RISETech for providing the technical infrastructure and resources that enabled the successful development and deployment of MediBuddy. Special thanks to the NUST School of Health Sciences (NSHS) for their collaboration and for supplying critical datasets and medical insights, which greatly enhanced the project's practical relevance.

Abstract

MediBuddy is an AI-powered diagnostic and educational platform designed to help medical professionals and students improve diagnostic accuracy and clinical decision-making. By leveraging large language models and role-based prompt engineering, the platform simulates realistic patient interactions, allowing users to take patient histories, interpret clinical signs, suggest investigations, and create management plans in an interactive environment. This hands-on approach helps users develop the skills necessary to handle a variety of clinical scenarios effectively.

The platform also features a chatbot for clinical dialogues, enabling users to practice patient communication and enhance their diagnostic reasoning. MediBuddy includes a case-based learning module with a diverse collection of case studies across different medical specialties, encouraging users to refine their clinical decision-making by testing their knowledge, proposing diagnoses, and recommending treatments. Interactive quizzes and assessments provide real-time feedback, supporting continuous learning and knowledge retention.

Accessible as a cloud-based web application, MediBuddy ensures flexibility for users by being available on any internet-enabled device. It also provides a robust API for easy integration into existing medical education systems or clinical environments. To ensure high-quality performance, MediBuddy's AI models are evaluated using metrics like BLEU and BERTScore, and their clinical relevance is validated through evaluations by practicing doctors, ensuring the platform aligns with real-world medical practices.

Contents

Acknowledgment	v
Abstract	vi
Contents	viii
List of Figures	ix
List of Tables	x
Chapter 1: Introduction	1
1.1 Motivation	1
1.2 Problem Statement	2
1.3 Scope	3
1.4 Aims and Objectives	5
1.5 Outcomes	6
1.6 Outcomes	7
Chapter 2: Literature Review & Background	9
2.1 BackGround of the study	9
2.2 Review of Existing Systems	10
2.2.1 Ada Health	10
2.2.2 Babylon Health	10
2.2.3 WebMD & Medscape	10
2.2.4 Lecturio, Amboss, and Osmosis	10
2.2.5 Osker AI	11
2.3 Research Gaps Identified	12
2.4 Summary of Literature Review	12
Chapter 3: Material & Component	13
3.1 Frontend	13
3.1.1 UI Libraries and Styling	13
3.1.2 Design Characteristics	14
3.2 Backend	14
3.2.1 Technologies and Frameworks	14
3.2.2 API Responsibilities	15
3.3 AI Model and NLP Layer	15
3.3.1 Language Model	15
3.3.2 Prompt Engineering Techniques	15
3.3.3 Evaluation and Feedback Mechanism	16
3.4 Authentication and User Management	16
3.5 Storage, Deployment, and Hosting	17
3.6 Admin Dashboard	17

3.7	Architecture Overview	17
3.8	Overview Table	18
Chapter 4: Methodology		19
4.1	Data Collection and Preprocessing	19
4.1.1	Initial sample DataSet: Joint Pain	20
4.1.2	Initial Data Upgradation	21
4.1.3	Latest & Extensive DataSet	21
4.2	AI Model Selection and Configuration	22
4.3	Frontend Design and Development	23
4.4	Backend Development and API Structure	23
4.4.1	Implemented API Endpoints	24
4.5	Evaluation and Testing Strategy	25
4.6	Project Timeline and Phases	26
4.7	System Workflow	27
Chapter 5: Deployment & Validation		29
5.1	Deployment Architecture and Infrastructure	29
5.1.1	Frontend Deployment via Netlify	29
5.1.2	Backend Deployment via Render	30
5.1.3	Model Hosting via Hugging Face Inference Endpoint	31
5.1.4	Inter-Service Integration	34
5.2	Validation and Performance Testing	34
5.2.1	Developer-Level Testing	34
5.2.2	Load and Stress Testing	35
5.2.3	Expert Validation with Doctors	35
5.2.4	Controlled User Testing with Medical Students	35
5.3	Conclusion	36
Chapter 6: User Interface and Screenshots		37
6.1	DashBoard	37
6.2	SignUp Page	39
6.3	Login Page	40
6.4	Disease Category Page	41
6.5	Patient List Page	41
6.6	Chatbot Interface	43
6.7	Scoring Area	45
6.8	Admin Panel (for Content Upload)	47
Chapter 7: Future Work & Conclusion		48
7.1	Integration of Voice Input and Output	48
7.2	Expanded Language Support	49
7.3	Visual Input and Diagnostic Imaging Support	50
7.4	Enhanced Admin and Instructor Portal	51
7.5	Conclusion	52
7.5.1	Summary of Contributions	52
7.5.2	Impact and Applications	53
7.5.3	Limitations	53
7.5.4	Final Thoughts	54
References		55

List of Figures

Figure 1	Sustainable Development Goals	ii
Figure 1.1	Annual Economic Losses by Sector in the United States	2
Figure 4.1	FastApi Endpoints	25
Figure 4.2	TimeLine of Project	27
Figure 4.3	Methodology	28
Figure 5.1	Deployed FrontEnd	30
Figure 5.2	GitHub	31
Figure 5.3	Deployment on render	32
Figure 5.4	Inference Endpoint	33
Figure 5.5	GPU Info of	33
Figure 6.1	DashBoard Hero Section	38
Figure 6.2	DashBoard features 2	39
Figure 6.3	DashBoard features 3	39
Figure 6.4	SignUp Page	40
Figure 6.5	Login Page	40
Figure 6.6	Categories Page	42
Figure 6.7	Patient List Page	43
Figure 6.8	ChatBot Interaction	45
Figure 6.9	Scoring Initial	47
Figure 6.10	Scoring Result	47
Figure 6.11	Admin DashBoard	47

List of Tables

Table 1	Range of Complex Problem Solving and Activities	iii
Table 2.1	Comparison of AI-Based Medical Tools	11
Table 3.1	Technological Components and Their Purpose in MediBuddy . . .	18

Chapter 1

Introduction

1.1 Motivation

In recent years, there has been an increasing recognition of diagnostic errors as a critical issue affecting patient safety and care quality. Medical students and young practitioners often struggle with integrating theoretical knowledge into practical diagnostic reasoning. Traditional learning methods, while effective in content delivery, sometimes fall short in simulating clinical complexity or providing personalized feedback. In response to this challenge, platforms like Osker AI have emerged, offering AI-based diagnostic assistance and training simulations. However, such solutions often come with significant limitations, such as paywalled features, restricted user interactions, and lack of integration into broader medical ecosystems.

MediBuddy was conceived as an effort to overcome these gaps by offering an open, modular, and extensible AI-powered platform. Its motivation lies in enhancing accessibility to advanced diagnostic simulations without sacrificing clinical quality. Moreover, with the integration of conversational AI, MediBuddy allows users to engage in natural, structured, and meaningful clinical conversations—thus bridging the gap between theoretical learning and practical application.

Additionally, the rise of generative AI tools like ChatGPT and their adaptation across in-

dustries inspired the idea of creating a domain-specific application that brings similar AI capabilities into the medical space. The potential to simulate real-time diagnostic workflows, offer structured case studies, and evaluate learning outcomes in a scalable digital format provides both pedagogical and technological motivation behind MediBuddy.

Moreover, the significance of improving diagnostic accuracy is not just a clinical or educational concern—it carries massive economic implications. According to recent data on annual economic losses in the United States, diagnostic mistakes alone account for approximately \$100 billion in losses each year. When considered alongside losses in other critical sectors such as finance (\$600B), retail (\$400B), and education (\$200B), it becomes evident that reducing medical errors could yield substantial national and systemic benefits. **(Refer to Figure 1.1)** for a visual representation of these figures.

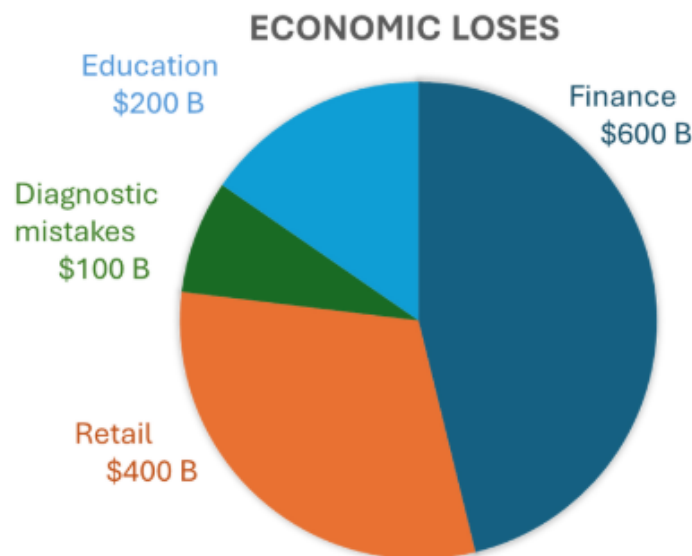


Figure 1.1: Annual Economic Losses by Sector in the United States

1.2 Problem Statement

Despite advancements in medical education and the proliferation of e-learning tools, there remains a significant gap in platforms that offer interactive, diagnostic simulations guided by clinical logic and real-time feedback. Most current solutions are either static, expen-

sive, or lack the contextual awareness required for nuanced clinical reasoning. Furthermore, junior healthcare professionals and students often struggle with forming differential diagnoses, interpreting signs and symptoms accurately, and integrating their decisions into coherent management plans.

The absence of an AI-powered, context-sensitive, and easily accessible diagnostic training platform poses a barrier to enhancing the clinical competencies of future healthcare providers. Additionally, limited collaboration between technical and medical domains has resulted in tools that either oversimplify medical workflows or fail to align with real-world practices.

MediBuddy aims to fill this gap by providing a robust, AI-powered platform that enables realistic doctor-patient simulations, supports diagnostic reasoning, and facilitates self-paced learning. The system is designed to be clinically relevant, technically sound, and scalable—allowing its adoption in both academic and professional healthcare settings.

1.3 Scope

The scope of the *MediBuddy* project encompasses the design, development, deployment, and evaluation of a web-based AI platform tailored for medical education and clinical simulation. This scope is defined to ensure a focused, measurable, and feasible implementation within the project’s timeframe, while also allowing future extensibility and integration with external healthcare and educational systems.

At its core, the project focuses on creating a domain-specific diagnostic chatbot capable of simulating doctor-patient interactions based on real medical case data. It leverages state-of-the-art large language models (LLMs) to deliver clinically meaningful, structured conversations that support the development of diagnostic reasoning in medical students and junior healthcare practitioners.

The system is designed to handle key components of a clinical consultation including

history taking, assessment of symptoms, differential diagnosis formation, test selection, and treatment planning. In addition to chatbot interaction, the platform also includes mechanisms for evaluation through preloaded doctor questions, user input tracking, and scoring mechanisms that can assess the quality of the interaction.

The technical scope includes:

- Development of a responsive and interactive front-end using React, integrated with APIs for real-time chatbot communication and feedback.
- Construction of a FastAPI-based backend that facilitates request handling, user management, scoring logic, and model communication.
- Deployment of the AI model (LLaMA 3.1 Aloe 8B) via Hugging Face's Inference Endpoints using scalable GCP infrastructure with NVIDIA L4 GPU support.
- Hosting of frontend and backend services using Netlify and Render respectively, with CI/CD pipelines enabled through GitHub integration.
- Inclusion of MongoDB for storing structured case data, user progress, scores, and authentication details.
- A modular structure that allows easy addition of new cases, medical specialties, question sets, or evaluation rubrics.

From a functional perspective, the scope is intentionally limited to non-invasive diagnostic simulation and educational reinforcement. It does not provide real-time patient diagnosis or replace clinical judgment. Instead, it is designed to enhance learning, reinforce best practices, and foster decision-making skills among users through realistic, AI-driven simulations.

The validation scope involves multi-stage testing, starting with internal developer-level testing, followed by clinical evaluation by experts from the NUST School of Health Sciences, and ultimately, controlled usage by medical students under supervised conditions.

These stages ensure that the product meets pedagogical, clinical, and usability standards before broader adoption.

In summary, the scope of MediBuddy spans technical development, AI integration, clinical simulation, and educational validation—offering a scalable foundation for future innovations in healthcare training and decision support.

1.4 Aims and Objectives

The primary aim of this project is to design and develop a web-based AI platform that simulates clinical scenarios and supports medical education through diagnostic reasoning, conversational practice, and case-based learning. To achieve this aim, the project sets out the following specific objectives:

- Design and deploy an AI chatbot powered by large language models capable of engaging in clinically meaningful conversations.
- Develop and integrate interactive learning modules such as quizzes, case studies, and simulations that assess and reinforce clinical decision-making.
- Build a RESTful API backend that supports seamless integration with external platforms such as hospital management systems or learning management systems (LMS).
- Ensure clinical validity and user-centric design through iterative evaluation involving medical professionals.
- Utilize cloud-based infrastructure for secure, scalable deployment of both frontend and backend services.
- Align the platform with global development goals related to healthcare, education, and innovation.

1.5 Outcomes

The development of MediBuddy has resulted in several key outcomes that span technical implementation, educational value, and practical applicability. These outcomes demonstrate the project's impact on both medical training and digital health innovation. The primary result is a fully developed, web-based platform that supports clinical simulations and interactive learning for medical users. The platform features an AI chatbot trained to handle structured clinical dialogues, capable of guiding users through history-taking, differential diagnosis, investigation recommendations, and treatment planning.

Further outcomes include:

- **Web-Based Diagnostic & Learning Platform:** A responsive and interactive platform deployed on the cloud, providing continuous access to training materials and diagnostic simulations.
- **AI-Powered Clinical Dialogue Agent:** A language model fine-tuned with role-based prompts to simulate patient interactions and guide learners through real-world medical workflows.
- **Interactive Educational Modules:** Tools such as structured quizzes and clinical case scenarios designed to improve critical thinking and diagnostic accuracy.
- **Modular Backend API:** A secure and scalable API interface for potential integration with third-party systems including telehealth platforms and academic LMS.
- **Expert Validation:** Clinical validation through collaboration with professionals at the NUST School of Health Sciences (NSHS), ensuring relevance and accuracy of case content.
- **Resource Support from RISETech:** Technical support in terms of hosting infrastructure, API handling, and cloud deployment facilitated by RISETech.

- **Evaluation via NLP Metrics:** Application of BLEU and BERTScore metrics to assess model outputs for grammatical quality and contextual relevance.
- **Sustainable Development Alignment:** Contribution to SDG 3 (Good Health and Well-being), SDG 4 (Quality Education), and SDG 9 (Industry, Innovation, and Infrastructure) by promoting innovation in healthcare education.

1.6 Outcomes

The development of *MediBuddy* has resulted in several key outcomes that span technical implementation, educational value, and practical applicability. These outcomes demonstrate the project's impact on both medical training and digital health innovation.

The primary result is a fully developed, web-based platform that supports clinical simulations and interactive learning for medical users. The platform features an AI chatbot trained to handle structured clinical dialogues, capable of guiding users through history-taking, differential diagnosis, investigation recommendations, and treatment planning.

Further outcomes include:

- **Web-Based Diagnostic & Learning Platform:** A responsive and interactive platform deployed on the cloud, providing continuous access to training materials and diagnostic simulations.
- **AI-Powered Clinical Dialogue Agent:** A language model fine-tuned with role-based prompts to simulate patient interactions and guide learners through real-world medical workflows.
- **Interactive Educational Modules:** Tools such as structured quizzes and clinical case scenarios designed to improve critical thinking and diagnostic accuracy.
- **Modular Backend API:** A secure and scalable API interface for potential integration with third-party systems including telehealth platforms and academic LMS.

- **Expert Validation:** Clinical validation through collaboration with professionals at the NUST School of Health Sciences (NSHS), ensuring relevance and accuracy of case content.
- **Resource Support from RISETech:** Technical support in terms of hosting infrastructure, API handling, and cloud deployment facilitated by RISETech.
- **Evaluation via NLP Metrics:** Application of BLEU and BERTScore metrics to assess model outputs for grammatical quality and contextual relevance.
- **Sustainable Development Alignment:** Contribution to SDG 3 (Good Health and Well-being), SDG 4 (Quality Education), and SDG 9 (Industry, Innovation, and Infrastructure) by promoting innovation in healthcare education.

Chapter 2

Literature Review & Background

2.1 BackGround of the study

The integration of artificial intelligence (AI) into the healthcare sector has significantly transformed diagnostic support, clinical workflows, and medical education. Tools powered by Natural Language Processing (NLP) and machine learning have introduced new possibilities for real-time interaction, intelligent symptom analysis, and adaptive learning systems. This progress is particularly relevant in medical education, where traditional teaching methods are often limited in their ability to simulate real-life clinical scenarios or provide dynamic, personalized feedback.

While numerous e-learning platforms and symptom checkers exist, few provide context-aware, interactive clinical reasoning or combine academic reinforcement with diagnostic training. This gap has led to a growing demand for tools that not only deliver medical content but also support decision-making and experiential learning through AI-driven interaction. The following review examines existing solutions and highlights the gaps that justify the development of a new system like MediBuddy.

2.2 Review of Existing Systems

Several AI-powered platforms currently offer services related to healthcare and medical learning. However, a critical examination reveals key limitations in their scope, adaptability, and interactivity.

2.2.1 Ada Health

Ada Health provides a user-friendly symptom checker based on a decision-tree model. It offers quick health assessments and general suggestions for users. However, it lacks the depth required for professional medical training and does not support conversational or interactive learning experiences.

2.2.2 Babylon Health

Babylon Health combines AI-powered assessments with telemedicine services, offering remote GP consultations. Although innovative, its AI reasoning lacks transparency, and its design does not cater to the academic or training needs of medical students seeking to understand diagnostic logic.

2.2.3 WebMD & Medscape

Both platforms serve as comprehensive references for symptoms, diseases, and treatment options. They are widely used by healthcare professionals and students, but their content is static and lacks the interactivity or adaptive learning features necessary for practical medical education.

2.2.4 Lecturio, Amboss, and Osmosis

These educational platforms are focused on structured exam preparation for medical students. They offer high-quality content, videos, and quizzes but lack the conversational

flexibility and real-time clinical simulation that would support diagnostic reasoning or role-play learning scenarios.

2.2.5 Osker AI

Osker AI is one of the few platforms that attempt to simulate clinical reasoning through AI-driven diagnostic exercises. It offers structured patient cases and interactive learning, primarily aimed at medical students. However, several limitations persist:

- Many features are paywalled, restricting access for wider educational use.
- Interactions are limited to predefined case structures, reducing flexibility.
- Personalization is minimal, and the system does not adapt dynamically to user level or previous performance.

Although Osker AI is a step forward, it does not offer a fully open, modular, and scalable environment that bridges both diagnostic support and medical education.

2.3 Comparative Analysis of Existing Systems

Table 2.1: Comparison of AI-Based Medical Tools

Platform	Focus Area	Strengths	Limitations
Ada Health	Symptom Checking	Simple interface, easy access	No academic depth or learning interaction
Babylon Health	Telemedicine + AI	Telehealth + AI fusion	Opaque decision-making, not education-focused
Amboss, Osmosis, Lecturio	Medical Education	High-quality structured content	Lacks real-time simulations and conversation
Osker AI	Clinical Simulation	Interactive case-based learning	Paywalls, limited interactivity, no integration

2.3 Research Gaps Identified

The literature and tools reviewed reveal several unmet needs:

- A lack of a unified AI system that supports both academic learning and diagnostic simulations.
- Existing platforms are often either too general or too rigid, limiting clinical reasoning development.
- Minimal support for context-aware, natural conversations between users and AI agents.
- Limited or no access to open, modular, and integrable systems for medical training.

These gaps justify the need for a platform like **MediBuddy** that combines interactive clinical simulation, diagnostic support, and educational reinforcement within a single AI-powered environment.

2.4 Summary of Literature Review

This chapter has highlighted the strengths and limitations of current tools in the AI-driven healthcare and medical education landscape. While platforms like Ada Health, Babylon, and Osher AI have made advancements in specific areas, none fully address the need for a conversational, open-access, diagnostic and educational assistant that is both flexible and clinically sound.

The identified gaps provide the foundation and justification for the development of MediBuddy — a system intended to bridge these divides by offering a more integrated, scalable, and intelligent solution for learners and professionals alike.

Chapter 3

Material & Component

MediBuddy is a full-stack, web-based clinical education platform designed to simulate real-world patient interactions using AI-driven conversational experiences. This chapter elaborates on the software tools, frameworks, models, deployment pipelines, infrastructure decisions, and architectural design that bring MediBuddy to life. The modular design supports scalability, secure operations, and flexibility for future enhancements.

3.1 Frontend

The frontend of MediBuddy is developed using **React.js**, a robust JavaScript library for building user interfaces, chosen for its component-based architecture, reusability, virtual DOM efficiency, and strong ecosystem support.

3.1.1 UI Libraries and Styling

- **Tailwind CSS** – A utility-first CSS framework employed for creating responsive, pixel-perfect UI components quickly and consistently.
- **React Icons** – Offers a large collection of customizable icons used throughout the application for medical symbols, user actions, and interface guidance.

- **React Router DOM** – Enables declarative, client-side routing for seamless transitions between user dashboard, chatbot interface, and admin panel.

3.1.2 Design Characteristics

- Consistent medical-themed color palette using shades of blue, white, and gray to maintain a clinical look and feel.
- Fully responsive design for desktop, tablet, and mobile compatibility using flex and grid layout systems.
- State management through React Hooks (useState, useEffect, useContext) and conditional rendering.
- Component-driven architecture for modular development and improved maintainability.

3.2 Backend

The backend is powered by **FastAPI**, a modern, high-performance Python web framework that enables efficient development of RESTful APIs using asynchronous capabilities and automatic documentation.

3.2.1 Technologies and Frameworks

- **FastAPI** – Provides asynchronous request handling, automatic data validation using Pydantic models, and interactive API documentation with Swagger UI.
- **MongoDB** – A document-based NoSQL database chosen for its flexibility in storing unstructured data like patient cases, user interactions, and admin inputs.
- **Uvicorn** – ASGI server used to run the FastAPI backend efficiently in production environments.

- **FastAPI-CORS Middleware** – Enables secure cross-origin resource sharing between the frontend and backend.

3.2.2 API Responsibilities

- Serve the React frontend routes during development and link client requests to appropriate AI and database services.
- Receive and process user messages and transmit them to the AI model.
- Fetch or update patient case information.
- Handle authentication requests and token validation.
- Store logs and user feedback for analytics and evaluation.

3.3 AI Model and NLP Layer

3.3.1 Language Model

MediBuddy integrates the **Meta LLaMA 3.1 Aloe** model via Hugging Face Inference Endpoints to simulate natural conversations based on clinical scenarios. The model was selected for its capability in handling nuanced medical prompts and producing context-aware, coherent responses.

3.3.2 Prompt Engineering Techniques

Prompt engineering is central to ensuring high-quality and realistic AI responses. Techniques used include:

- Embedding patient-specific data (age, gender, chief complaint, history) at the start of the conversation.

- Using delimiters and structured formatting to separate system instructions from user inputs.
- Applying few-shot learning patterns to simulate expected questioning styles or clinical reasoning.
- Reinforcing behavior constraints to minimize hallucinations and avoid diagnostic or therapeutic misinformation.

3.3.3 Evaluation and Feedback Mechanism

BERTScore, a transformer-based evaluation metric, is used to semantically compare the student's inputs reference data provided by DataBase. This enables:

- Reflective learning based on qualitative alignment rather than keyword matching.
- Scoring of free-text responses using semantic embeddings.

This evaluate both user question asked from AI patient about his case with reference questions and at the end the diagnosis (disease identified), treatment plan and reason for diagnosis is also evaluated and on the basis of all of these a score was assigned to our medical user.

3.4 Authentication and User Management

- **JWT (JSON Web Tokens)** – Stateless, secure user authentication across frontend and backend. Used for protecting user-specific routes and handling sessions.
- User roles (admin, student) are encoded in the token payload to manage access control.
- Token expiry and refresh logic implemented for session security.

3.5 Storage, Deployment, and Hosting

- **Render.com** – Hosts the FastAPI backend server with auto-deployment from GitHub. Provides scalability, secure environment variables, and free SSL.
- **Hugging Face Inference Endpoints** – Secure, production-grade deployment of the LLaMA model with latency optimization and resource allocation controls.
- **Netlify** – Used for frontend deployment in early development phases due to its simple CI/CD pipeline and free-tier support.

Additional deployment considerations included:

- CORS configuration between Netlify (frontend) and Render (backend).
- Endpoint security through HTTPS and authentication headers.
- Environment separation for development and production modes.

3.6 Admin Dashboard

- Built in React.js with protected routes accessible only to authenticated admin users.
- Allows entry of patient demographics, presenting complaints, lab findings, diagnosis, and treatment plans.
- Submissions are validated and then inserted into MongoDB.
- Includes options to update or delete cases that was previously by administration.

3.7 Architecture Overview

- **Frontend:** React-based application with dynamic routing and responsive UI.

- **Backend:** FastAPI API layer for routing, data validation, and asynchronous I/O handling.
- **Database:** MongoDB Atlas instance for structured and semi-structured data.
- **AI Engine:** LLaMA 3.1 Aloe model via Hugging Face Inference Endpoint.
- **Evaluation:** BERTScore using sentence embeddings.
- **Authentication:** JWT token-based access control.
- **Deployment:** Render (backend), Hugging Face (AI model), Netlify (frontend prototype).

3.8 Overview Table

Table 3.1: Technological Components and Their Purpose in MediBuddy

Component	Technology	Reason
Frontend	React	Efficient, component-based JavaScript library for building responsive web interfaces
Backend	FastAPI	Modern, fast (high-performance) web framework for building APIs with Python
Chatbot Model	LLaMA 3.1 Aloe	Domain-specific natural language processing for clinical and diagnostic dialogue
Database	MongoDB	NoSQL database supporting flexible and schema-less relational data storage
Design	Figma	Collaborative prototyping and UI design tool used for wireframing and iteration
Hosting	Render / Vercel	Modern hosting platforms providing seamless deployment and CI/CD integration

Chapter 4

Methodology

This chapter presents a comprehensive walkthrough of the development methodology used in building MediBuddy. It outlines how data was gathered and processed, how AI and backend services were structured, and the timeline-based evolution of the system through iterative testing, integration, and deployment.

4.1 Data Collection and Preprocessing

The project began with a dataset provided by NSHS containing 20 pediatric cases in Microsoft Word format. Each case included rich, clinically accurate data, such as:

- Patient name and demographic details
- Presenting complaints and full medical history
- Family medical background
- Clinical examination findings (e.g., fever, blood pressure, etc.)
- Diagnostic test results

4.1.1 Initial sample DataSet: Joint Pain

Description: Tommy, a 14-year-old, presents with left knee pain persisting for two months. The pain occurs after football matches or training, described as soreness to touch with mild, occasionally noticeable swelling. No fever, redness, or movement restriction is reported, and the pain resolves with rest. He applies ice post-activity but is unsure of its effectiveness. He is concerned about a possible fracture and its impact on his football season and requests an X-ray. His development is average in height, with recent growth over the past six months.

Past Medical History: History of mild eczema in childhood.

Family History: No relevant findings.

Drug History & Allergies: No known drug allergies. Took paracetamol during severe pain episodes.

Social History: Plays for the school U15A football team and takes it seriously. Enjoys gym workouts.

Investigations: Examination shows normal gait, slight swelling over the left tibial tuberosity, and localized tenderness. No pain on passive movement. No further investigations needed. Diagnosis: Osgood-Schlatter disease. Differentials include osteomyelitis, Perthes disease, SUFE, muscle strain, fracture, septic arthritis, reactive arthritis, juvenile idiopathic arthritis, Ewing sarcoma, and osteosarcoma.

Management: Recommend paracetamol or NSAIDs for pain relief. Apply intermittent ice packs over the tibial tuberosity (10–15 minutes, up to 3 times daily, especially post-exercise). Use protective knee pads and reduce or stop sporting activities. Suggest alternative activities like gym workouts to stay active. Stretching exercises may help; refer to the Victorian Paediatric Orthopaedic Network for a factsheet.

Viva: Reactive arthritis usually follows an extra-articular infection such as Salmonella,

Shigella, or Campylobacter. Septic arthritis differs with more acute onset, severe pain (even on passive movement), joint swelling, and systemic symptoms like fever. Common organisms include *Staphylococcus aureus* and *Neisseria gonorrhoeae*. Developmental dysplasia of the hip is the orthopaedic condition screened for during routine neonatal examinations.

4.1.2 Initial Data Upgradation

To adapt this data to the needs of the AI model, we transformed each case into a structured paragraph format. This allowed for easier integration into a prompt-based AI interaction system. During this early phase, we used this limited dataset to prototype the system architecture and develop the basic features of the frontend and backend.

4.1.3 Latest & Extensive DataSet

As the application matured, we expanded our dataset using open-source medical case repositories hosted on **Hugging Face**. These additional cases enriched the system's diversity and clinical depth. This DataSet is in A comprehensive Python-based data engineering pipeline was developed to reformat and sanitize the Hugging Face data. This included extracting patient narratives, cleaning textual inconsistencies, and aligning the data schema with our internal format.

The refined dataset now includes:

- Structured clinical case narratives.
- patient name, intro and key info of Patient.
- Real-life doctor questions that will be used for evaluation of user questions.
- Reference diagnosis and treatment plans.
- Category of Patient Disease

These augmentations enabled the integration of a robust scoring and grading system by providing a gold standard for evaluating student responses using semantic similarity and context alignment.

4.2 AI Model Selection and Configuration

We evaluated multiple state-of-the-art language models, including different versions of **LLaMA**, **DeepSeek**, and other open-weight LLMs. Our criteria included their capacity to handle domain-specific medical terminology, inference speed, and compatibility with deployment constraints.

After a month of rigorous empirical testing on diverse clinical prompts, we finalized the use of **LLaMA 3.1 Aloe 8B** for its:

- Deep understanding of medical case context
- High-quality and coherent response generation
- Efficient model size-to-performance trade-off

Rather than fine-tuning the model, we adopted a robust **prompt engineering** approach. This allowed us to simulate various medical learning scenarios and control AI behavior through carefully designed prompt templates. Each template included structured inputs such as patient information, guiding instructions, response constraints, and dynamic scoring cues.

This method enabled us to iteratively fine-tune the AI outputs without retraining, while optimizing performance and contextual relevance through experimentation over hundreds of test cases.

4.3 Frontend Design and Development

The user interface was designed using **Figma** during the October–November period. The design phase focused on user-centric principles, aiming to provide an intuitive, responsive, and educationally effective platform.

Frontend development was carried out using **React.js**, supplemented with Tailwind CSS for UI styling and React Router for seamless navigation.

Key frontend modules include:

- A real-time chatbot interface that facilitates user-AI interaction
- An admin dashboard for managing medical cases and monitoring responses
- Integrated scoring and feedback presentation for learner evaluation
- User authentication and session management

The frontend was tested across different screen sizes to ensure optimal performance on desktop, tablet, and mobile devices.

4.4 Backend Development and API Structure

The backend was developed using **FastAPI** due to its high performance, support for asynchronous processing, and automatic API documentation. FastAPI's integration with Pydantic enabled strict data validation, crucial for handling structured medical data.

Core backend services include:

- Chat interaction endpoints linked to the AI model
- Case data management and storage
- Evaluation pipeline for scoring user inputs
- Session handling, context updates, and history clearing

4.4.1 Implemented API Endpoints

- **GET /Med_cases** — Fetch list of available patient cases.
- **POST /** — Main AI interaction endpoint for chat input.
- **POST /res** — Handles response logging.
- **POST /qa** — Generate question answer according to case selected.
- **POST /quiz** — Generate MCQs according to case selected.
- **POST /clear_history** — Clears ongoing conversation history and context selected.
- **POST /set_context** — Sets the AI context using patient data.
- **POST /addcase** — Admin tool to add new patient cases to the database.
- **POST /score** — Evaluates/scores the questions asked by user.
- **POST /finalScore** — Return the total score, overall evaluation of the case.
- **POST /compare** — Measures similarity score between two text i.e. user and reference responses.
- **POST /set_ques** — Set questions from which user questions will be compared.
- **POST /clear_ques** — Clears questions from which user questions will be compared.

MongoDB was used as the primary database to store case data, user sessions. This DataBase can be updated by admin portal when new cases are added. Data is provided on BackEnd according to endpoints calls. The figure 4.1 shows all of the FASTAPI endpoints that can be used, tested and integrated to FrontEnd.

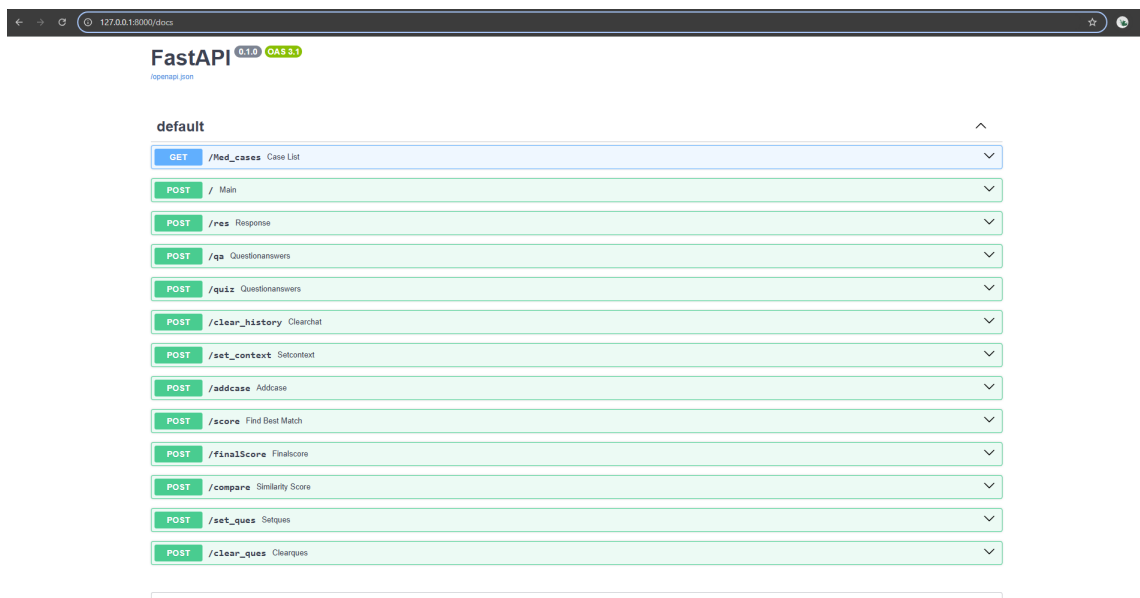


Figure 4.1: FastApi Endpoints

4.5 Evaluation and Testing Strategy

To validate MediBuddy’s functionality and learning effectiveness, we adopted a multi-tier testing strategy.

Initial testing was conducted by the development team using simulated cases to verify application logic, UI functionality, and backend endpoint reliability.

Subsequently, expert validation was performed by doctors from the NUST School of Health Sciences. They reviewed the AI-generated responses for clinical accuracy, appropriateness, and educational value. Their feedback played a critical role in refining prompts, scoring mechanisms, and feedback quality.

In the final stage, the application was released to a controlled group of medical students. This cohort interacted with the chatbot in realistic exam-like scenarios. Feedback was gathered via embedded forms and usage analytics, which helped identify areas for improvement in:

- Chat experience quality

- Accuracy and clarity of AI responses
- Feedback usefulness
- Case diversity and challenge level

These insights guided further prompt refinement, scoring threshold adjustments, and UI tweaks for a more educational experience.

4.6 Project Timeline and Phases

The development of MediBuddy followed a structured timeline with overlapping phases to allow parallel task completion:

- **Design Phase:** October – November (UI mockups, system architecture diagrams)
- **Frontend Development:** November – January (React components, routing, styling)
- **Backend Development:** November – February (FastAPI endpoints, MongoDB schemas)
- **Integration:** January – April (Frontend-backend-AI integration)
- **Deployment:** January – April (Render, Netlify, Hugging Face Inference Endpoint)
- **RAG Implementation:** February – April (Context retrieval and injection into prompts)
- **Authentication:** February – March (JWT-based login, user roles)
- **Feedback Generation:** February – March (AI-based feedback and evaluation)
- **Testing and QA:** February – May (Internal and external testing, bug fixing)

GitHub was used for source control and collaboration, while deployment was managed through CI/CD pipelines for efficient versioning and rollout.

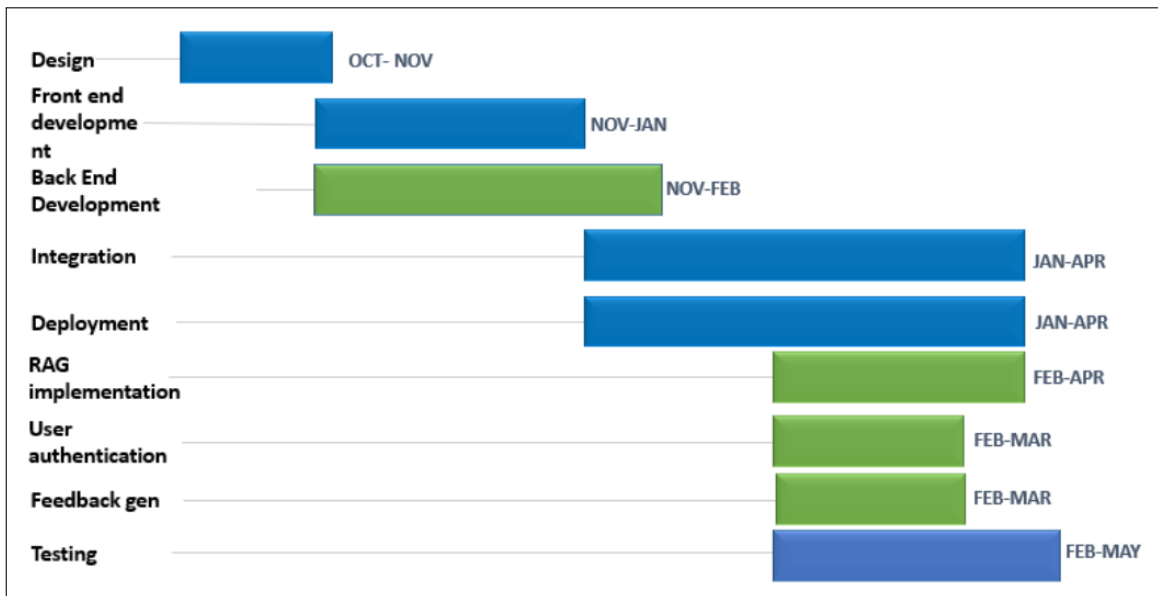


Figure 4.2: TimeLine of Project

4.7 System Workflow

The system workflow of **MediBuddy** involves multiple interconnected modules that operate seamlessly to deliver a responsive and intelligent pediatric medical assistant. The workflow can be divided into the following stages:

1. **User Interaction Layer** Users (patients or medical students) interact with the system through a web-based frontend built using React. The interface allows them to input symptoms, ask questions, or simulate OSCE-style clinical scenarios.
2. **API Request Handling (Backend)** User input is sent to a FastAPI backend via HTTP requests. FastAPI efficiently handles routing and asynchronous communication.
3. **Preprocessing and Context Handling** The backend performs input preprocessing, including tokenization and context preservation (using past dialogue history stored in MongoDB).
4. **Model Inference via HuggingFace Transformers** The preprocessed input is passed to a fine-tuned biomedical language model (BioBERT, LLaMA-based Aloe model)

hosted locally or via the HuggingFace Transformers library. The model generates an appropriate response based on the context and medical knowledge.

5. **Postprocessing and Response Structuring** The raw output is formatted into human-readable text. Medical terminology or abbreviations may be simplified for non-clinical users (optional).
6. **Database Logging (MongoDB)** Each interaction (including input, response, timestamps, and user ID) is stored in MongoDB for future retrieval, auditability, and analysis.
7. **Response Delivery** The generated response is sent back to the frontend, where it is displayed to the user in real time.
8. **Evaluation and Feedback (Optional)** For internal testing or user studies, evaluation metrics such as BERTScore can be used to assess model performance and refine outputs.

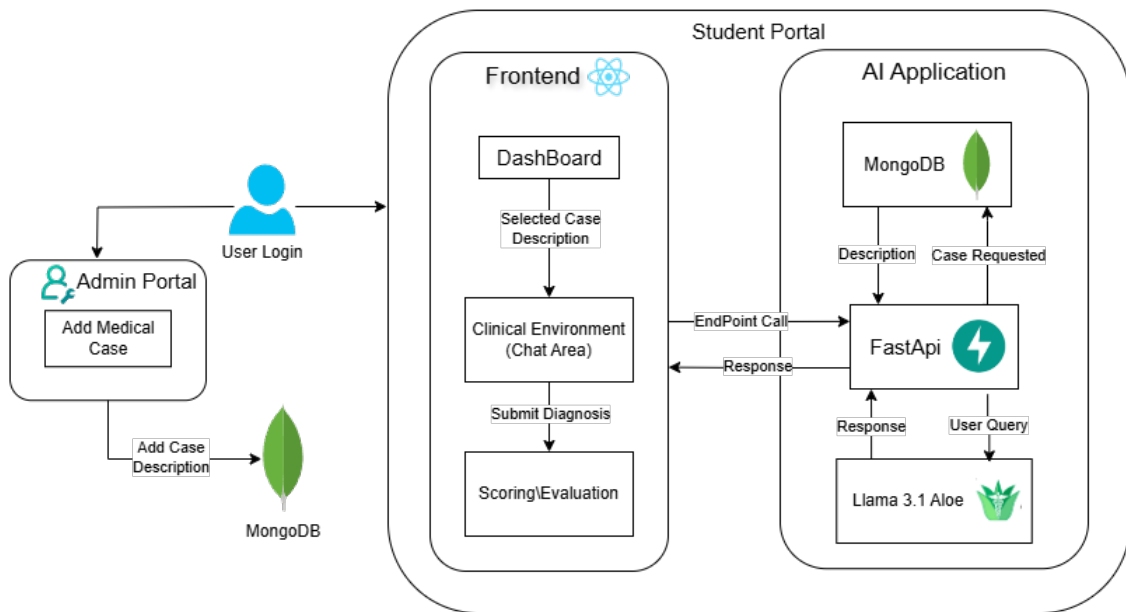


Figure 4.3: Methodology

Chapter 5

Deployment & Validation

This chapter discusses in detail the technical infrastructure, deployment strategies, and multi-phase validation processes that ensure the robust performance, scalability, and educational impact of MediBuddy.

5.1 Deployment Architecture and Infrastructure

5.1.1 Frontend Deployment via Netlify

The frontend of MediBuddy was developed using **React.js** and designed with modern UI/UX principles using Figma. For deployment, **Netlify** was selected due to its efficient and developer-friendly platform for hosting modern web applications. Netlify provides an intuitive interface, excellent integration with GitHub, and automatic CI/CD pipelines, allowing developers to push changes directly from version control to a live environment with minimal overhead.

The integration with GitHub streamlined our workflow—every new push to the production branch would trigger an automatic deployment on Netlify. Furthermore, Netlify offers:

- A global Content Delivery Network (CDN) ensuring fast load times and perfor-

mance globally

- Version control and instant rollback functionality
- Built-in HTTPS with automatic SSL certificates for secure browsing
- Custom domain mapping
- Automated preview environments for every pull request

These features enabled seamless and efficient frontend deployment and iterative development with real-time feedback.

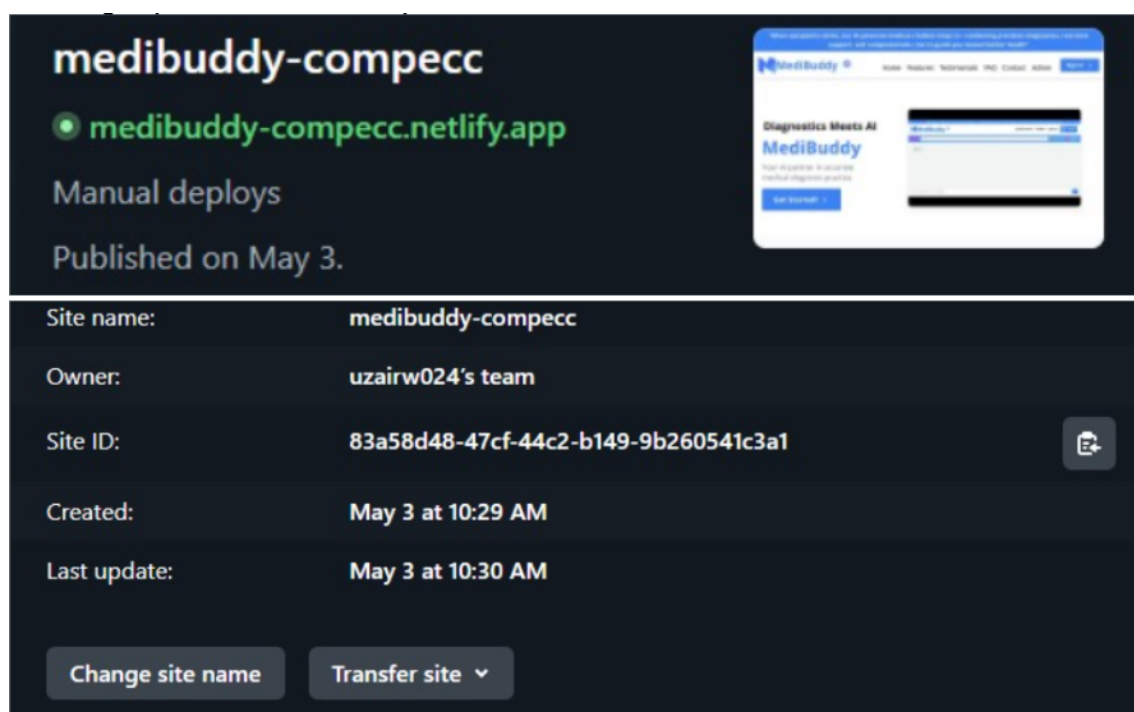


Figure 5.1: Deployed FrontEnd

5.1.2 Backend Deployment via Render

The backend services were architected using **FastAPI**, chosen for its speed, asynchronous capabilities, and intuitive design suited for RESTful API creation. Deployment was handled via **Render**, a modern cloud platform that supports scalable web service hosting.

Render's integration with GitHub made it possible to deploy the backend with a single

click. Each key version of the backend codebase was pushed to GitHub and deployed automatically. Although we utilized the free tier of Render for this project, it was sufficient for development, testing, and early production stages. Benefits included:

- Git-based CI/CD with automated builds and deployments
- Native support for Python/FastAPI environments
- Environment variable management through the dashboard
- Access to real-time logs and status pages

This setup enabled agile development with minimal deployment friction, ensuring continuous integration and continuous delivery (CI/CD) workflows were maintained throughout the development cycle.

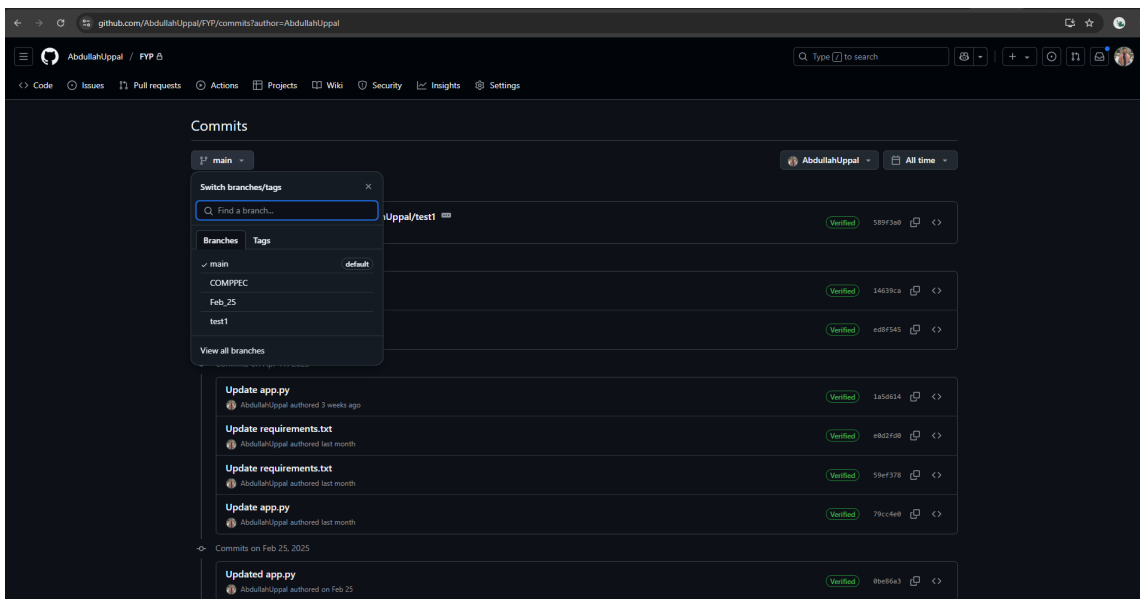


Figure 5.2: GitHub

5.1.3 Model Hosting via Hugging Face Inference Endpoint

Our core AI component—**LLaMA 3.1 Aloe 8B**—was deployed on **Hugging Face Inference Endpoints**, leveraging **Google Cloud Platform (GCP)** infrastructure and the **NVIDIA L4 24GB GPU**. This setup was carefully selected for its balance of perfor-

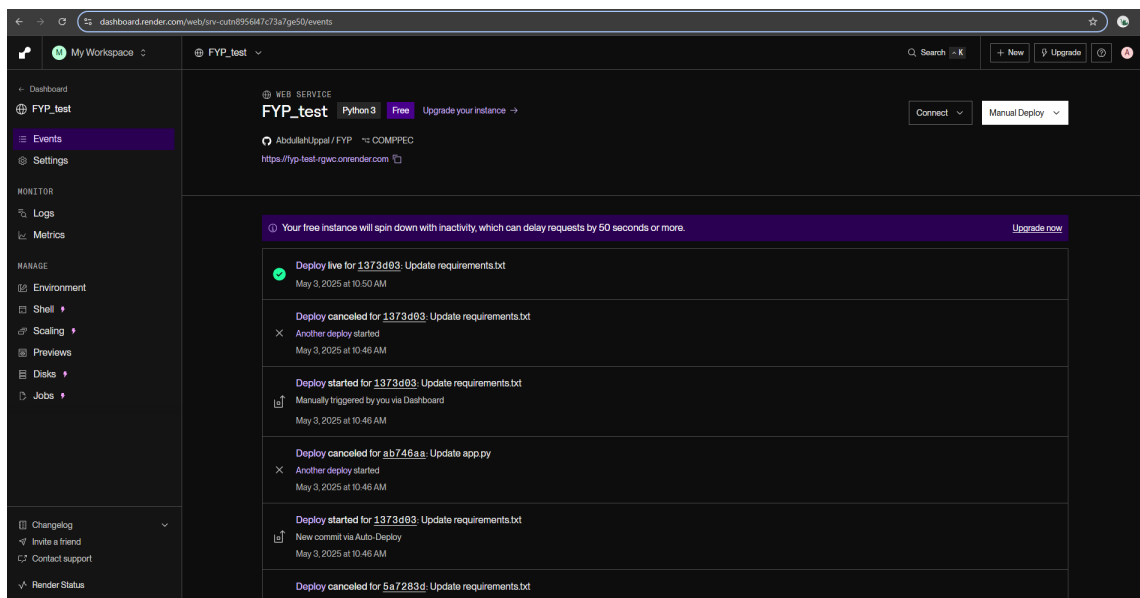


Figure 5.3: Deployment on render

mance, cost, and auto-scaling capabilities.

Key highlights of this model deployment include:

- **Auto-scaling:** Automatically scales to zero after 15 minutes of inactivity, saving GPU costs. It scales up automatically when new queries arrive.
- **Resource Optimization:** GCP infrastructure provides GPU-accelerated inference with a pay-as-you-go model at \$0.70 per hour.
- **Monitoring Dashboards:** Real-time monitoring tools are provided, including visualizations of query load, performance stats, and inference logs.
- **Secure API Integration:** Access is controlled through API keys, and detailed logs are maintained for every model request.

The backend serves as a middle layer, managing all user interactions, formatting prompts, querying the AI model, and handling responses. This design modularity allows for performance optimization, error logging, and scalability without interfering with the frontend experience.

Inference Endpoints Dashboard | Catalog | Help

Back to Dashboard

llama3-1-aloe-beta-8b-hbw Scaled to Zero Pause Wake up

Overview | Analytics | Usage & Cost | Logs | Settings

Endpoint has been scaled to zero due to inactivity.
Use the button above or send a request to wake it up. Note: scaled-to-zero Endpoints still count towards quota usage.

Endpoint URL | Need help?
https://xe197nhez34ohfy5.us-east4.gcp.endpoints.huggingface.cloud

Configuration | Protected

question-answering | TGI Container

Created Feb 20 by AbdullahUppal • Last Edited Feb 25 at 2:58 PM by proxy

Container URI | Port: 80
ghcr.io/huggingface/text-generation-inference:3.1.0

Model | Up-to-date
HPAI-BSC/Llama3.1-Aloe-Beta-8B

Revision 44d4f4c

Instance | \$0.7/h while running
GCP | us-east4 | GPU: Nvidia L4 · 1x GPU · 24 GB

Scale-to-zero After 15 minutes with no activity

Figure 5.4: Inference Endpoint

Back to Dashboard

llama3-1-aloe-beta-8b-hbw Paused Resume

Overview | Analytics | Logs | Usage & Cost | **Settings**

Hardware Configuration | Request

Amazon Web Services | Microsoft Azure | Google Cloud Platform

CPU | **GPU** | US East 4 | us-east4

GPU Model	Configuration	Cost
Nvidia T4	1 GPU · 16 GB 3 vCPUs · 11 GB	\$0.5/h
Nvidia L4	1 GPU · 24 GB 3 vCPUs · 12 GB	\$0.7/h
Nvidia A100	1 GPU · 80 GB 11 vCPUs · 174 GB	\$3.6/h
Nvidia L4	4 GPUs · 96 GB 46 vCPUs · 186 GB	\$3.8/h
Nvidia A100	2 GPUs · 160 GB 23 vCPUs · 348 GB	\$7.2/h
Nvidia H100	1 GPU · 80 GB 25 vCPUs · 240 GB	\$10/h
Nvidia A100	4 GPUs · 320 GB 47 vCPUs · 696 GB	\$14.4/h
Nvidia H100	2 GPUs · 160 GB 51 vCPUs · 480 GB	\$20/h
Nvidia H100	4 GPUs · 320 GB 102 vCPUs · 960 GB	\$40/h

Update Endpoint

\$0.70 / h
per running replica

Figure 5.5: GPU Info of

5.1.4 Inter-Service Integration

All components of the system—frontend (Netlify), backend (Render), and model (Hugging Face Inference Endpoint)—are interconnected through RESTful API architecture. This modular and loosely-coupled integration ensures:

- Better maintainability and flexibility
- Efficient request routing and error handling
- Scalability for each module independently

This architecture enhances system reliability, facilitates quick debugging, and allows the introduction of new features without downtime or architecture-wide changes.

5.2 Validation and Performance Testing

5.2.1 Developer-Level Testing

During early development stages, extensive developer testing was conducted to identify issues in logic, model behavior, and system integration. The developers focused on:

- Functional testing of all API endpoints
- Manual edge-case testing with adversarial inputs to probe the model’s reasoning boundaries
- Continuous prompt refinement to reduce hallucinations and improve contextual alignment
- Frontend responsiveness across devices and screen sizes

Testing iterations were rapid and guided by logs and user feedback. As bugs or unexpected behaviors were identified, updates were made to either prompts or the FastAPI logic, and redeployments were carried out immediately.

5.2.2 Load and Stress Testing

To assess system performance under real-world usage, simulated load testing was implemented. Multiple users were simulated to evaluate:

- Backend’s ability to handle concurrent requests without timeouts or crashes
- Model endpoint’s scalability and response time under increasing loads
- End-to-end latency across the system pipeline

This phase confirmed that the system could support multiple concurrent users and that the autoscaling of inference endpoints worked as expected.

5.2.3 Expert Validation with Doctors

Upon functional completion and internal testing, the platform was handed over to clinical experts and faculty from the **NUST School of Health Sciences**. Their tasks involved:

- Evaluating AI-generated responses for clinical correctness
- Assessing scoring algorithms and diagnosis suggestions
- Recommending enhancements for clearer explanations and reduced ambiguity

Based on this feedback, modifications were made to the backend scoring logic, new disease categories were added, and further refinements were made to prompt design.

5.2.4 Controlled User Testing with Medical Students

Following expert validation, a controlled trial was conducted with a group of medical students. Each student was asked to engage with all available patient cases and interact with the chatbot in diagnostic mode.

User feedback was collected both quantitatively (through forms) and qualitatively (via interviews). They evaluated:

- Ease of navigation and user experience
- Clarity and helpfulness of AI-generated feedback
- Accuracy and fairness of performance scoring
- Educational value and realism of case scenarios

This round of testing helped fine-tune the UI, scoring thresholds, and prompt wording. It also validated that MediBuddy could effectively support medical education in a semi-formal learning context.

5.3 Conclusion

This chapter elaborated on the robust, scalable deployment strategy of MediBuddy across Netlify, Render, and Hugging Face Inference Endpoints. The modular integration allowed rapid updates, while monitoring tools provided visibility into application performance. Rigorous validation—first by developers, then doctors, and finally end-users—ensured the reliability and educational accuracy of the application, making it suitable for clinical learning environments.

Chapter 6

User Interface and Screenshots

A critical part of **MediBuddy**'s success lies in its intuitive, elegant user interface tailored to the workflows of students and clinicians. This chapter presents screenshots and visual descriptions of the application.

6.1 DashBoard

The landing page serves as the first point of interaction for users, setting the tone and providing an overview of MediBuddy's offerings. It combines functionality, aesthetics, and accessibility to cater to both students and administrators.

- **Home Area (Hero Section):** This prominent area introduces MediBuddy's mission and capabilities:
 - **Platform Title and Tagline** – e.g., “MediBuddy – Your AI-Powered Clinical Tutor.”
 - **Brief Description** – A concise summary of how MediBuddy supports clinical reasoning, diagnostics, and academic growth through intelligent simulations.
 - **Demo Video Embed** – An autoplay or user-initiated video walkthrough that

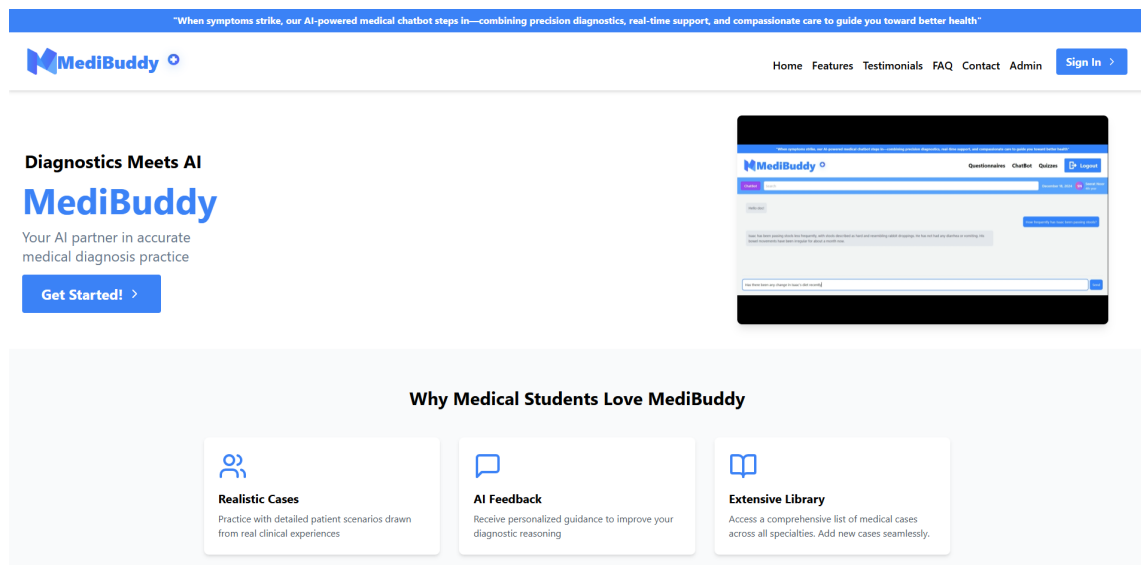


Figure 6.1: DashBoard Hero Section

visually demonstrates a typical student interaction with the platform, covering login, chatbot use, and quiz attempts.

- **Call-to-Action (CTA) Button** – Options such as “Get Started,” “SignIN,” or “Admin Portal.”
- **Navigation Bar:** Positioned at the top of the page, it provides quick access to the most important sections of the Dashboard. Key links include:
 - **Student Login and Admin Login** – redirects users to their respective portals for personalized access.
 - **Features** – outlines the core modules like chatbot, quiz engine, educational resources, and performance tracking.
 - **Testimonials** – displays student and educator feedback to build trust and showcase real-world impact.
 - **FAQ** – addresses common questions related to usage, technical issues, and learning scope.
 - **Contact Us** – includes email, phone, and a message form for support or col-

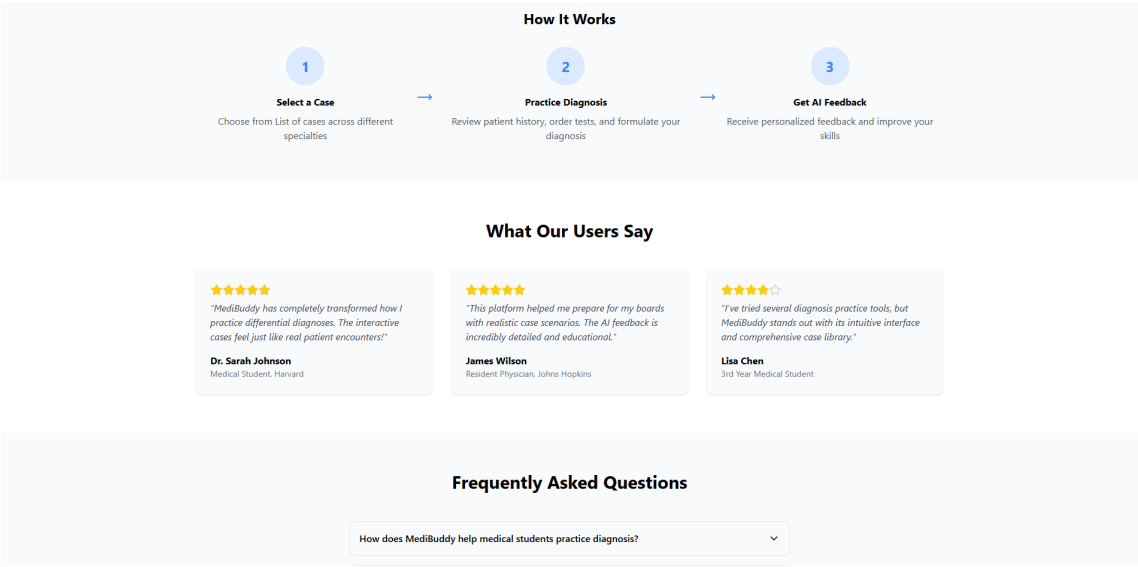


Figure 6.2: DashBoard features 2

laboration inquiries.

- **Email Subscription** – allows visitors to subscribe to updates, news, and product announcements.

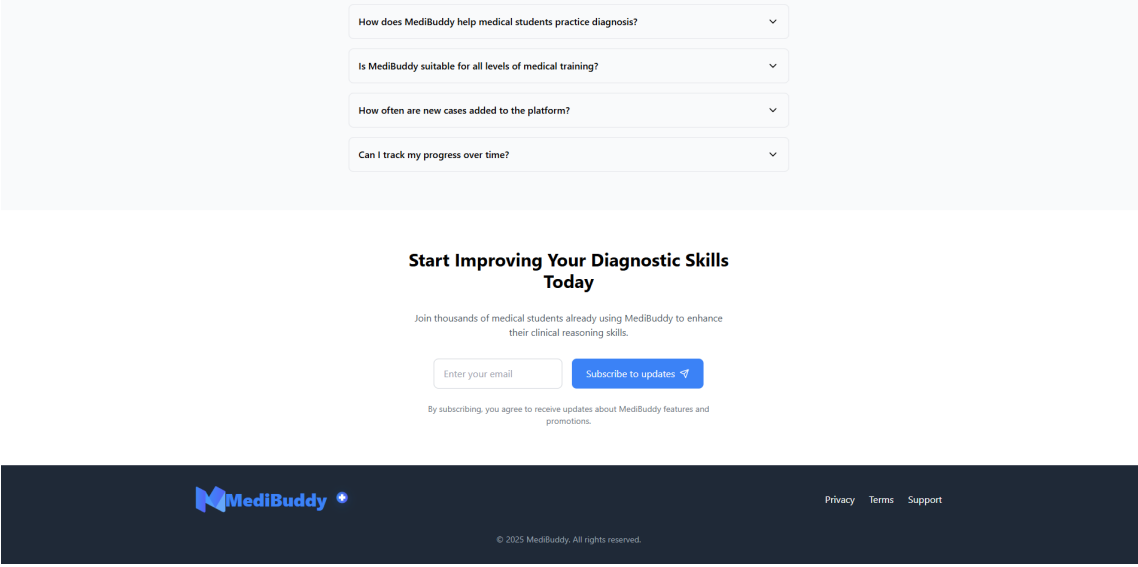
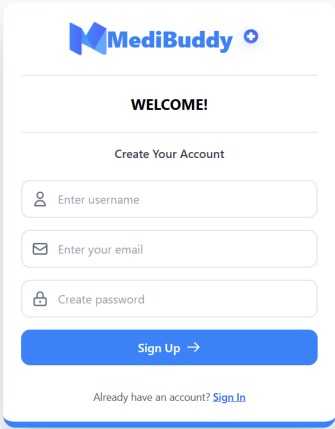


Figure 6.3: DashBoard features 3

6.2 SignUp Page

- **UI Theme:** Clean, light-colored with a medical blue palette.

- **Features:** Email SignUp, Google sign-up, checking password complexity.
- **Feedback:** Instant error messages for invalid format and error.

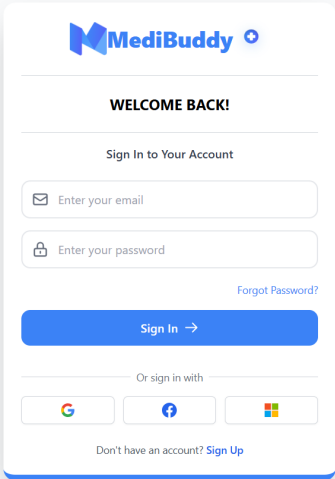


The image shows a mobile app sign-up screen for MediBuddy. At the top is the MediBuddy logo. Below it is a 'WELCOME!' message. The main heading is 'Create Your Account'. There are three input fields: 'Enter username' with a person icon, 'Enter your email' with an envelope icon, and 'Create password' with a lock icon. A blue 'Sign Up →' button is below the fields. At the bottom, it says 'Already have an account? [Sign In](#)'.

Figure 6.4: SignUp Page

6.3 Login Page

- **UI Theme:** Clean, light-colored with a medical blue palette.
- **Features:** Email login, Google sign-in, forgot password.



The image shows a mobile app login screen for MediBuddy. At the top is the MediBuddy logo. Below it is a 'WELCOME BACK!' message. The main heading is 'Sign In to Your Account'. There are two input fields: 'Enter your email' with an envelope icon and 'Enter your password' with a lock icon. A blue 'Sign In →' button is below the fields. To the right of the password field is a link 'Forgot Password?'. Below the button, it says 'Or sign in with' followed by three social media icons: Google, Facebook, and Microsoft. At the bottom, it says 'Don't have an account? [Sign Up](#)'.

Figure 6.5: Login Page

- **Feedback:** Instant error messages for invalid login.

6.4 Disease Category Page

After a successful login, the user is redirected to the Disease Category Page. This serves as the entry point for exploring various medical conditions and initiating patient case simulations.

- **User Interface:**

- The screen displays a grid layout of clickable disease categories such as *Cardiology*, *Pediatrics*, *Dermatology*, *Gastroenterology*, etc.
- Each category is represented by a card containing a title, a brief description, and a relevant icon or image.

- **Functionality:**

- Clicking on a category navigates the user to the Patient List Page where clinical cases related to the selected category are displayed.
- Categories can be filtered by difficulty level (e.g., beginner, intermediate, advanced) or case type (e.g., acute vs chronic).

- **Purpose:**

- Organizing cases by system or specialty improves usability and allows focused practice in a specific area of medicine.

6.5 Patient List Page

Once a user selects a disease category, they are directed to the Patient List Page, where multiple patient cases from the selected category are available for interaction.

- **Layout:**

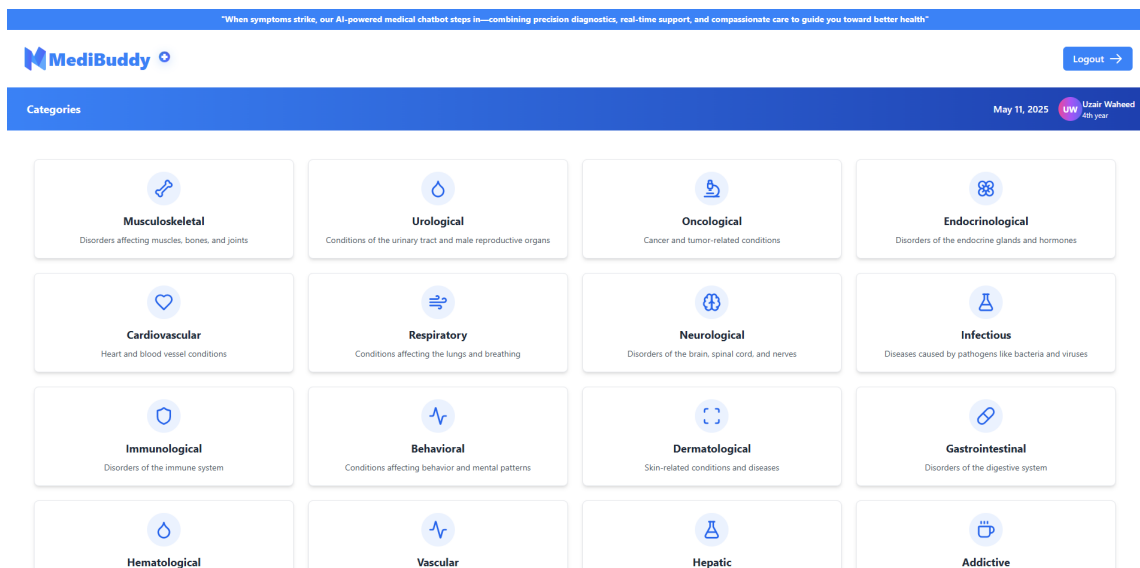


Figure 6.6: Categories Page

- Patients are displayed in a card-based grid or list format.
- Each patient card includes:
 - * Patient Name (e.g., “Mr. Ahmed, 45”)
 - * Basic Demographics (age, gender)
 - * Chief Complaint (e.g., “Chest pain for 2 hours”)
 - * A “Start Diagnosis” button

• **Interaction:**

- Users can review the case summaries and select any patient to begin the simulated diagnostic session with the chatbot interface.
- A search bar and filters (e.g., by age group, complexity) help narrow down the case list.

• **Learning Objective:**

- This layout allows users to choose cases that match their learning needs or challenge level.

- Encourages self-directed exploration of diverse clinical presentations within each category.

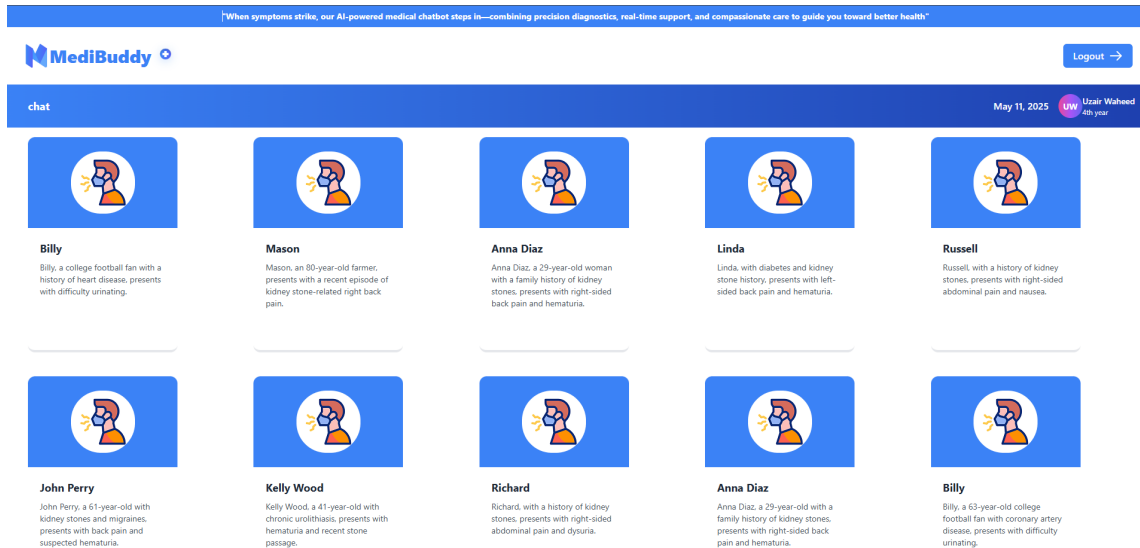


Figure 6.7: Patient List Page

6.6 Chatbot Interface

The chatbot interface serves as the core functional space where users engage in simulated clinical conversations. Designed with clarity and realism in mind, it mimics interactions between a medical student and a virtual patient, allowing users to practice clinical questioning, reasoning, and diagnosis in a safe, responsive environment.

- **Conversational UI with Speech Bubbles:**

- Messages appear in a threaded bubble format, with user inputs on the right and the chatbot's responses (representing a virtual patient) on the left.
- Color-coded bubbles help distinguish between user and system messages for better readability.

- **Text Input Area:**

- Located at the bottom of the interface, this is where users type their questions

or clinical prompts.

- Supports auto-focus, placeholder text (e.g., "Ask your patient about symptoms..."), and keyboard shortcuts for fast interaction.
- No media upload or multimedia features to preserve clinical realism and focus on verbal questioning.
- Includes a "Send" button and also supports Enter-key submission.

- **Real-Time Virtual Patient Responses:**

- The chatbot responds instantly using natural language generation to simulate patient answers based on underlying clinical logic.
- Responses are context-aware, meaning they change depending on previous inputs and clinical history being developed.
- Supports simulated behaviors such as uncertainty, clarification questions, or emotional expressions for realism.

- **Scrollable Chat History:**

- Entire conversation is preserved in a scrollable pane for reference.
- Useful for reviewing earlier questions and adjusting diagnostic thinking accordingly.
- Includes timestamps to indicate interaction flow.

- **Case-Based Context Initialization:**

- Each chatbot session can be tied to a specific case (e.g., chest pain, pediatric fever, skin rash), which defines the initial context and background of the virtual patient.
- Contextual info may include basic demographics (age, gender) and the reason for visit (chief complaint).

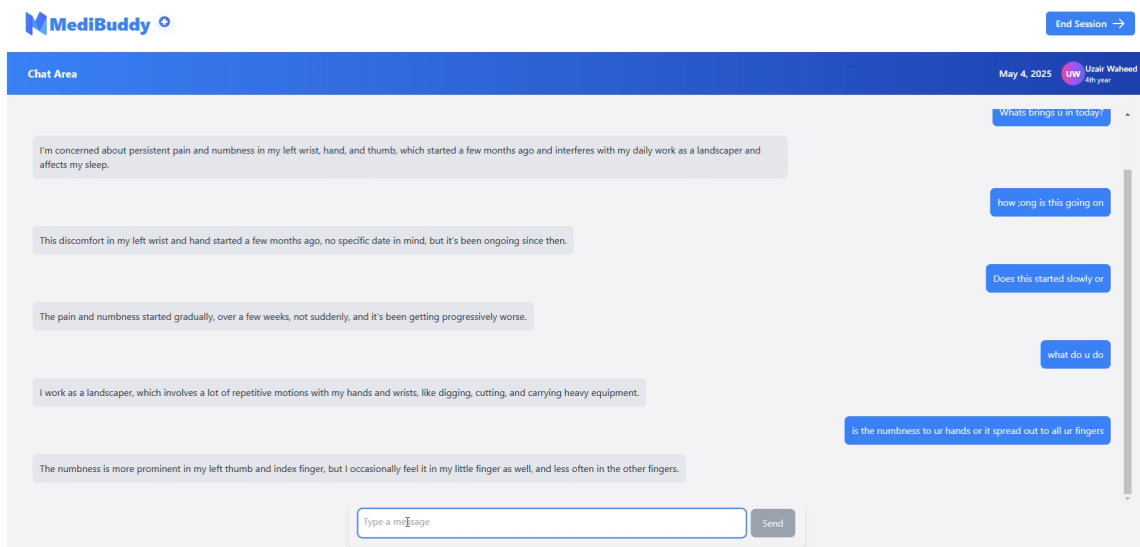


Figure 6.8: ChatBot Interaction

- **Ending chat:** At the end of the session, users select end case and scoring screen pops up.

6.7 Scoring Area

The scoring area is activated at the end of a chatbot session when the user selects the “End Case” option. This section is designed to assess the user’s clinical performance and reinforce learning through structured reflection and automated evaluation.

- **Activation and Transition:**
 - Upon selecting the *End Case* button, the chatbot interface is hidden and the scoring panel becomes visible.
 - A brief summary of the virtual patient’s case is displayed at the top, including contextual information such as age, gender, and chief complaint.
- **Input Fields for Clinical Reasoning:**
 - **Text Area 1 – Key Clinical Issue:** Users are prompted to summarize the core problem or diagnosis they have identified during the session.

- **Text Area 2 – Reasoning and Management Plan:** A space for users to elaborate on their diagnostic reasoning, treatment plan, suggested investigations, and any recommended medications.
- These fields encourage critical reflection and help instructors or the AI system gauge depth of understanding.
- **Submission and Feedback:**
 - After completing both text fields, the user clicks the *Submit* button.
 - On submission, the input areas disappear, and a performance score is generated based on:
 - * Relevance and depth of questions asked during the chat.
 - * Clinical accuracy of the diagnosis.
 - * Appropriateness of the reasoning and treatment plan.
 - A breakdown of the score by category (History Taking, Reasoning, Management) is presented visually using bars or percentages.
- **Navigation and Continuity:**
 - After viewing the score, the *Submit* button is replaced by a new button labeled *Return to Categories*.
 - Clicking this navigates the user back to the patient case categories page, where they can choose a new case for practice.

Medical Evaluation

Diagnosis *

Carpal tunnel syndrome

Assessment & Treatment Plan *

symptoms have not improved with these conservative measures, we will discuss further treatment options including additional diagnostic testing or possible surgical intervention. The patient states

+ Save Medical Evaluation

Figure 6.9: Scoring Initial

Case Evaluation Results

You are Correct!

4.63/10

Needs improvement

Overall Score 46%

Return to Categories

Figure 6.10: Scoring Result

6.8 Admin Panel (for Content Upload)

- Dashboard view: Contains a form with different sections for adding a new patient case in dataSet.
- New Patient case will get added in database directly.

MediBuddy

Admin Dashboard

May 11, 2025 Admin System Administrator

Medical Case

Patient Name

Enter patient name

Patient Info

Enter 2-3 line summary about patient

Case Description

Describe the medical case in detail...

Category

Enter Disease Category

Enter diagnosis/Treatment plan

Enter diagnosis of disease

Enter dialogues of Doctor

+ Add Medical Case

Figure 6.11: Admin Dashboard

Chapter 7

Future Work & Conclusion

While MediBuddy has achieved its core objectives in intelligent medical assistance, diagnostic training, and academic reinforcement, the scope for future improvements and enhancements is vast. This chapter outlines potential extensions and areas for future development to improve usability, accuracy, and impact.

7.1 Integration of Voice Input and Output

Currently, **MediBuddy** operates in a text-based input-output mode. Future versions can include multimodal interaction capabilities to enhance accessibility and user experience. Proposed improvements include:

- **Speech-to-Text (STT) Integration:** Incorporating STT technologies via APIs such as Google Speech-to-Text, Whisper by OpenAI, or Mozilla DeepSpeech will allow users to input queries using natural spoken language. This is especially helpful for users who are not comfortable typing or have motor impairments.
- **Text-to-Speech (TTS) Conversion:** Using TTS engines such as Google Cloud TTS, Amazon Polly, or open-source alternatives like Coqui TTS to convert the AI's responses into audible speech. This will benefit visually impaired users and

enhance hands-free usability in clinical settings.

- **Voice UX Optimization:** Designing conversational flows and UI elements that support back-and-forth spoken interactions with minimal latency and confusion, potentially using Voice Activity Detection (VAD) to manage turn-taking.
- **Audio Feedback Customization:** Allowing users to choose voice types, pitch, speed, and language preferences to ensure personalized and comfortable interaction.

These enhancements aim to make the system more inclusive and practical for real-world clinical environments and underserved populations.

7.2 Expanded Language Support

To reach a wider and more diverse audience, particularly in multilingual regions, support for multiple languages should be integrated into **MediBuddy**. This expansion would help break down language barriers in healthcare education and diagnostics. Possible approaches include:

- **Support for Regional Languages:** Adding natural language support for Urdu, Arabic, Pashto, Punjabi, and other regional languages relevant to the target population. This will increase usability and trust among non-English speakers.
- **Multilingual NLP Models:** Leveraging pretrained multilingual models like mBERT (Multilingual BERT), XLM-R (XLM-RoBERTa), and MBART to understand and generate text across multiple languages.
- **Real-time Translation APIs:** Integrating APIs such as Google Translate or Azure Cognitive Services for on-the-fly translation of queries and responses while maintaining medical terminology accuracy.
- **Localized Medical Terminology:** Curating domain-specific lexicons and ontologies in local languages to ensure accurate interpretation and generation of medical

content.

- **Dialect and Accent Adaptation:** Using fine-tuned models to adapt to regional dialects and variations in spoken input to improve recognition and understanding.

Multilingual support will significantly broaden MediBuddy's accessibility and societal impact by democratizing medical knowledge and assistance.

7.3 Visual Input and Diagnostic Imaging Support

Many medical conditions—such as dermatological issues, eye infections, throat inflammation, and lymph node swelling—require visual examination for accurate diagnosis and clinical reasoning. To improve diagnostic precision and simulate real-world consultation scenarios, future versions of **MediBuddy** should integrate visual data support. This extension can be approached through the following strategies:

- **Image Dataset Integration:** Incorporate publicly available medical image datasets (e.g., HAM10000 for skin lesions, EyePACS for diabetic retinopathy, or the ISIC archive) to allow the AI to learn from validated diagnostic visuals.
- **Custom Image Generation:** Use generative models like DALL·E, Stable Diffusion, or other domain-specific image generation tools to create illustrative images for specific symptoms or conditions. These can simulate case visuals for training or educational use.
- **User-Uploaded Images (Future):** In later versions, allow users to upload their own images (e.g., of the eye, skin, throat) for AI-based assessment. This would require integrating image classification models trained on annotated medical images, while ensuring robust data privacy measures.
- **Visual-AI Fusion:** Combine NLP with computer vision using multimodal models to interpret both text and image inputs for a comprehensive diagnosis, e.g., "red eye

+ fever" with image-based conjunctivitis identification.

- **Visual Case Bank:** Build a structured visual case library mapped to symptom clusters and diseases. This would serve as a visual learning tool, especially for academic reinforcement and OSCE-style training.

Adding visual input capabilities not only enhances diagnostic realism but also strengthens the educational value of MediBuddy. It brings the platform closer to the real-world diagnostic workflows used by healthcare professionals.

7.4 Enhanced Admin and Instructor Portal

To support academic institutions and educators in effectively monitoring and guiding students, the development of a robust and data-rich admin or teacher portal is essential. This portal should enable instructors to track individual and group performance, identify common errors, and customize teaching strategies accordingly. Future enhancements to the **MediBuddy** admin module could include:

- **Student Progress Dashboards:** Provide visual summaries of each student's learning journey, including session logs, completed cases, and topic-wise coverage.
- **Mistake Tracking and Analysis:** Automatically log and categorize student errors (e.g., diagnostic missteps, missed differentials, incorrect terminology). Highlight frequent mistake patterns across cohorts for curriculum feedback.
- **Scoring and Evaluation:** Assign real-time scores to each case or simulation based on medical reasoning, diagnostic accuracy, and interaction quality. Allow customizable rubrics to suit academic or clinical standards.
- **Comparative Analytics:** Display performance benchmarks, class averages, and individual comparisons to motivate students and help educators identify outliers needing support.

- **Instructor Feedback System:** Enable teachers to add manual comments, personalized feedback, or corrective guidance on specific student cases.
- **Export and Reporting Tools:** Generate downloadable performance reports for individual students or entire classes in CSV or PDF format for academic use.
- **Access Control and Role Management:** Allow multiple admin or faculty roles with different levels of access, such as curriculum designers, observers, or evaluators.

These features would significantly enhance the academic utility of MediBuddy by turning it into not just a learning tool, but a performance-monitoring and decision-support system for medical educators.

7.5 Conclusion

This project has presented the design, development, and evaluation of **MediBuddy** — an AI-driven medical assistant aimed at supporting pediatric diagnosis, education, and healthcare management. Built upon state-of-the-art language models such as BioBERT and LLaMA, and integrated with robust APIs and user-centric interfaces, the system successfully demonstrates how artificial intelligence can be effectively applied in real-world medical scenarios.

7.5.1 Summary of Contributions

The core contributions of this project are summarized below:

- Developed a robust, modular AI-powered medical chatbot using transformer-based NLP models fine-tuned for biomedical contexts.
- Integrated FastAPI for scalable backend services and React for a dynamic frontend experience.

- Utilized MongoDB for efficient storage and retrieval of user data and diagnostic logs.
- Incorporated diverse datasets and evaluation metrics (e.g., BERTScore) to validate the model's relevance and performance.
- Addressed usability in a medical education setting through simulated OSCE scenarios and dialogue modeling.

7.5.2 Impact and Applications

The system has broad implications in both clinical and educational settings:

- **Clinical Use:** MediBuddy offers rapid preliminary support for medical diagnosis, which can reduce human error and speed up decision-making in busy healthcare environments.
- **Medical Education:** The chatbot's ability to simulate OSCE-style questioning offers a unique, interactive platform for training medical students and evaluating clinical reasoning.
- **Cost Implications:** As highlighted in the economic loss graph (to be added), AI-assisted systems like MediBuddy could help mitigate diagnostic and operational inefficiencies—potentially saving billions annually in sectors like healthcare, education, and diagnostics.

7.5.3 Limitations

Despite its success, MediBuddy has some limitations:

- **Model Dependency:** The system is limited by the scope and biases of its underlying language models and training datasets.
- **Lack of Real-time Clinical Data:** It currently does not integrate with hospital

EMRs or real-time patient data.

- **Evaluation Scope:** While BERTScore and dialogue-based evaluations provide some insights, broader clinical trials and expert feedback are necessary for real-world deployment.

7.5.4 Final Thoughts

This project serves as a foundation for future work in intelligent clinical assistants and medical training tools. It illustrates the vast potential of language models and AI in improving healthcare delivery and medical education. With continued improvements in model accuracy, ethical safeguards, and real-world testing, systems like MediBuddy could play a critical role in the future of medicine.

References

- [1] Vaswani, A., Shazeer, N., Parmar, N., et al. *Attention is All You Need*. Advances in Neural Information Processing Systems (NeurIPS), 2017. arxiv:1706.03762
- [2] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. Proceedings of ACL, 2019. arxiv:1810.04805
- [3] Lee, J., Yoon, W., Kim, S., et al. *BioBERT: A pre-trained biomedical language representation model for biomedical text mining*. Bioinformatics, 36(4), 1234–1240, 2020. doi:10.1093/bioinformatics/btz682
- [4] Liu, P., Li, W., Xu, Y., et al. *Evaluating LLMs on Medical Reasoning and Knowledge*. arXiv preprint, 2023. arxiv:2305.09617
- [5] Zhang, S., Liu, Q., Li, Y., et al. *LLM-Eval: Unified Evaluation of LLMs across Tasks and Benchmarks*. arXiv preprint, 2023. arxiv:2305.10923
- [6] Wolf, T., Debut, L., Sanh, V., et al. *Transformers: State-of-the-art Natural Language Processing*. Proceedings of EMNLP: System Demonstrations, 2020. doi:10.18653/v1/2020.emnlp-demos.6
- [7] Tiangolo, S. *FastAPI: Modern Web Framework for Building APIs with Python 3.7+*. <https://fastapi.tiangolo.com>

- [8] React Team. *React: A JavaScript Library for Building User Interfaces*. Meta Platforms Inc. <https://react.dev>
- [9] MongoDB Inc. *MongoDB Developer Documentation*. <https://www.mongodb.com/docs/>
- [10] Figma Inc. *Figma – Collaborative Interface Design Tool*. <https://www.figma.com>
- [11] HuggingFace Model: HPAI-BSC/Llama3.1-Aloe-Beta-8B. <https://huggingface.co/HPAI-BSC/Llama3.1-Aloe-Beta-8B>
- [12] Zhang, T., Kishore, V., Wu, F., et al. *BERTScore: Evaluating Text Generation with BERT*. ICLR, 2020. arxiv:1904.09675
- [13] Streamlit Inc. *Streamlit – Turn Data Scripts into Shareable Web Apps*. <https://streamlit.io>
- [14] Chen, H. *Medical Dialogues and Notes Dataset – Hugging-Face*. <https://huggingface.co/datasets/chenhaodev/medical-dialogs-notes>
- [15] Paediatrics OSCE Case Bank. *Internal Training Resource*, 2024.
- [16] Norman, D. A., & Draper, S. W. *User Centered System Design: New Perspectives on Human-Computer Interaction*. CRC Press, 1986.
- [17] Nielsen, J. *Usability Engineering*. Academic Press, 1994.